

EC8202 Big Data Analytics

Smart Grid Energy Monitoring & Billing System

Report

Group Members:

Horadagoda H.A.D.A.M. - EG/2021/4557

Madhusankha T.G.S.A. - EG/2021/4656

Pathirathna A.H. - EG/2021/4705

Contents

1 Introduction and Business Requirements 1
1.1 Non-functional requirements and assumptions 1
2 Architecture Decision 1
2.1 Selected architecture: Lambda-style hybrid 1
2.1.1 Accepted Design 1
2.1.2 Rejected alternative 1
3 System Architecture and Data Flow 2
3.1 Real-Time data path 2
3.2 Batch data path 2
3.3 Synchronization semantics 2
4 Technology Selection and Justification 3
5 Data Sources and Ingestion 3
5.1 Smart-meter reading model 3
5.2 Event schema and derived value 4
5.3 Kafka topic and partitions 4
5.4 Daily tariff source 4
6 Stream Processing with Apache Spark 4
6.1 Validation and parsing 4
6.2 Zone window aggregation 4
6.3 Daily household aggregation 4
6.4 Checkpointing and restart behavior 4
7 Batch Orchestration with Apache Airflow 5
7.1 Tariff-ingestion DAG 5
7.2 Daily-billing DAG 5
7.3 Failure scenarios 5
8 Storage, Serving and Presentation 5
8.1 Relational storage model 5
8.2 API layer 5
8.3 Dashboard 5
9 Observability, Reliability and Testing 6
9.1 What is observed and why 6
9.2 Automated tests 6
9.3 Reliability controls 6
10 Results and Evaluation 6
10.1 Demonstrated outputs 6
10.2 Evaluation against requirements 8
11 Limitations, Trade-offs and Production Improvements 8

11.1 Current limitations 8
11.2 Production-scale improvements 9
12 Conclusion 9
13 Group Contribution Statement 9

List of Figures

Figure 1: End-to-end Lambda-style architecture and major data flows. 2
Figure 2: Durable per-date readiness and billing state transitions. 2
Figure 3: Energy-performance dashboard populated from PostgreSQL through FastAPI after stream processing. 7
Figure 4: Bill values visualized in the dashboard after DAG runs 7
Figure 5: Control Center for System Monitoring and light weight control 8

1 Introduction and Business Requirements

1.1 Non-functional requirements and assumptions

The design focuses on reproducibility, data accuracy, recoverability and demonstrability on an 8 GB development computer. Docker Compose packages the stateful services, while the host runs lightweight producers, API and dashboard. PostgreSQL constraints and transactional writes protect derived data. Spark checkpoints preserve stream offsets and aggregation state.

Simulation starts at 2026-01-01 00:00 UTC. Twenty household readings are emitted per real second, one reading per household. Time advances by 288 simulated seconds for every real second, therefore a simulated day is completed in 300 real seconds.

2 Architecture Decision

2.1 Selected architecture: Lambda-style hybrid

2.1.1 Accepted Design

The system that we built, uses a Lambda-style architecture because it contains a real-time constraint path and a batch or schedulable data path. The real-time speed path includes smart-meter events that are continuously contained and processed through Kafka and Spark. The batch path includes validation of daily tariff files and performs daily billing through Airflow. PostgreSQL joins the outcomes through a durable readiness record. This is a practical Lambda interpretation. It separates workloads by latency and data semantics without deliberately maintaining two copies of every calculation.

Decision factor	**Lambda-style response**
Latency	Zone conditions are updated from the live meter stream without waiting for a complete day or tariff file.
Reference data	Tariffs are bounded files with strict completeness rules. So, a scheduled batch DAG is more suitable.
Correctness	Billing is withheld until final daily energy and required daily tariff files exist in the database for the same date.
Recovery	The real-time and batch components may stop independently. Reliable offsets, checkpoints and readiness state can guarantee later continuation.

2.1.2 Rejected alternative

A Kappa design would treat the event log as the main computational source and would normally reproduce corrections by replaying the stream through one processing model. That approach is valid when all significant inputs can be expressed as events and when one stream-processing codebase should serve both live and historical computation. It was not selected here because the tariff source is explicitly a daily generated file and billing task can only be done if both tariff data and aggregated energy data for the given day are available. Converting every tariff file, validation result and billing command into a replayable event workflow would require a higher storage capacity to store replayed data and additional complexity without improving the business outcome.

3 System Architecture and Data Flow

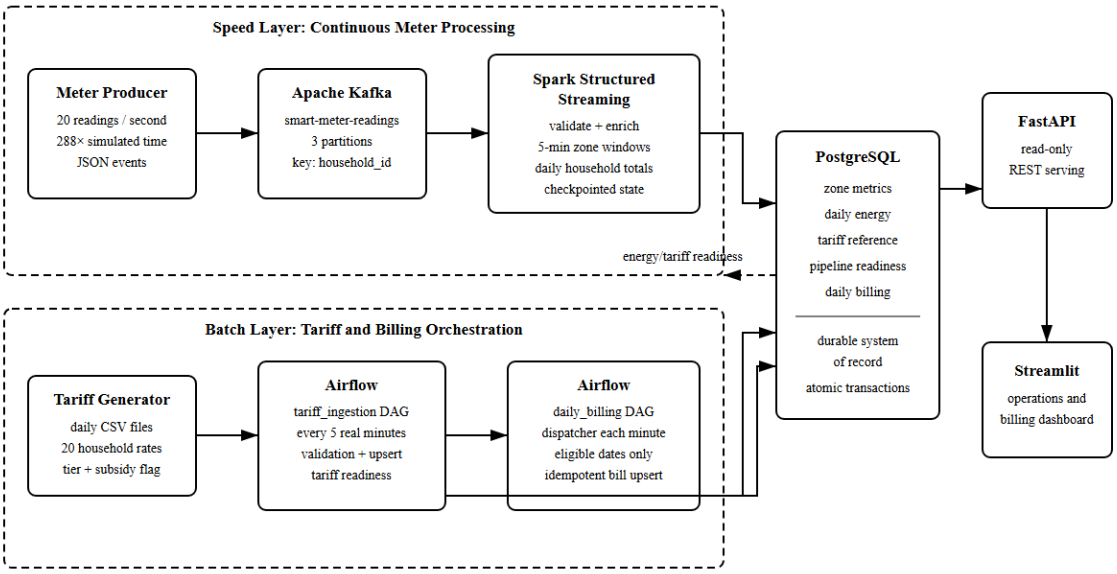


Figure 1: End-to-end Lambda-style architecture and major data flows.

3.1 Real-Time data path

The smart-meter producer creates one reading for each household at every real-second and publishes the twenty records to Kafka. The message key is the household identifier. Kafka hashes the key to one of three partitions, ensuring that events from the same household will be in the same partition while distributing different households across partitions. Spark subscribes to the topic and parses the JSON payload using an explicit schema. Invalid rows are removed before aggregation.

Spark calculates grid import for each event, groups valid readings into five-minute event-time windows by grid zone and writes the resulting metrics to PostgreSQL database. A second streaming aggregation groups readings by household and simulated date. A ten-minute watermark bounds late-event state. When a day is final according to the watermark, Spark writes all daily household totals and marks energy readiness in the same database transaction.

3.2 Batch data path

The tariff generator creates a CSV for each simulated date. The tariff-ingestion DAG scans for files every five real minutes. It validates the exact schema, row count, identifier uniqueness and rate validity before uploading tariff records to the database. It then marks tariff readiness for the file's date in the same transaction. The billing DAG is a one-minute dispatcher. It queries PostgreSQL for any date whose energy and tariff flags are both true and whose billing state is pending or failed.

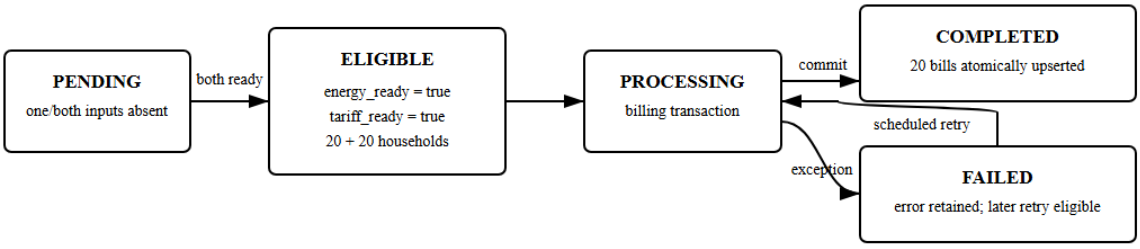


Figure 2: Durable per-date readiness and billing state transitions.

3.3 Synchronization semantics

Airflow does not listen directly to an in-memory Spark notification. Such a signal could be lost while Airflow is unavailable. Instead, Spark publishes the ready event by updating the database. Airflow's next scheduled dispatch reads all eligible dates, including dates completed while it was stopped. Therefore, the schedule determines detection latency, not data eligibility or correctness. This design also handles out-of-order source completion: tariffs may arrive before energy, or energy may arrive before tariffs.

4 Technology Selection and Justification

Layer	Selected technology	Justification and rejected option
Ingestion	Apache Kafka 3.9	Kafka matches the required producer/topic/partition model, decouples producer and consumer availability, provides ordered partition logs and retains events for recovery. Direct producer-to-Spark communication was rejected because it tightly couples their lifecycles and removes the replayable buffer.
Streaming	Apache Spark Structured Streaming 3.5.1	Spark is preferable to Storm here because the team can use a unified declarative model and checkpointed state with less topology-specific code. Storm could provide low per-event latency, but the five-minute analytical windows do not require it. Since this is not a time critical task even though real-time data is used, Spark is suitable.
Orchestration	Apache Airflow 3.3.1	Airflow makes file validation, readiness selection, retries and billing dependencies visible as scheduled DAGs. It is used for bounded batch work.
Storage/sink	PostgreSQL 16	The serving model is compact, relational and consistency-sensitive. SQL joins and atomic transactions naturally the required computation for bill calculation. Cassandra is less suitable for these joins and multi-table transactions. HDFS/S3 with Parquet would improve large historical scans but would add operational weight and would not replace the transactional serving database at this scale.
Serving	FastAPI	A typed read-only REST layer prevents the dashboard from embedding database credentials and offers automatic endpoint documentation.
Presentation	Streamlit	It provides a lightweight Python dashboard for KPIs, charts, date selection and bills, keeping effort focused on data engineering rather than front-end infrastructure.
Packaging	Docker Compose and uv	Compose declares reproducible stateful services and volumes; uv locks Python dependencies and creates an isolated environment. Full Kubernetes orchestration would be excessive.

5 Data Sources and Ingestion

5.1 Smart-meter reading model

The readings are synthetic but are not arbitrary independent numbers. Each household is assigned a fixed meter, grid zone and base demand. Zones A, B and C receive 7, 7 and 6 households respectively. Thirteen households have rooftop solar. A fixed seed makes household profiles reproducible between runs.

Base electricity consumption is sampled between 0.15 and 0.45 kWh per reading. It is shaped by a time dependent multiplier representing common residential demand. Here, we are using 0.6 from midnight to 05:00, 0.8 from 05:00 to 06:00, 1.6 from 06:00 to 09:00, 1.0 from 09:00 to 18:00, 2.0 from 18:00 to 22:00 and 1.2 from 22:00 to midnight. Each value also receives a random factor between 0.85 and 1.15. This prevents identical repeated measurements.

Solar households have a capacity between 0.10 and 0.50 kWh per reading. Generation follows a daylight curve with sunrise at 06:00, a peak near 12:00 and sunset at 18:00, multiplied by a simulated weather factor between 0.75 and 1.0. Non-solar households report zero generation. This model produces explainable daily demand peaks and renewable variation while remaining lightweight.

5.2 Event schema and derived value

Field	Type	Meaning
meter_id	string	Stable physical meter identifier.
household_id	string	Logical household and Kafka message key.
consumption_kwh	number	Electricity consumed during the simulated interval.
solar_generation_kwh	number	On-site solar energy generated during the interval.
grid_zone	string	Zone A, B or C used for operational aggregation.
timestamp	ISO-8601 UTC string	Simulated event time, independent from processing time.

Grid import is derived downstream rather than generated independently:

Grid import = Maximum value of (Grid Energy Consumption - Solar Generation) and 0.

5.3 Kafka topic and partitions

The Kafka topic used in our project is *smart-meter-readings* that has three partitions. Partitioning enables parallel consumption and spreads storage, but does not imply that every partition receives exactly the same number of events. Kafka guarantees order inside a partition rather than across the complete topic. Keying by household ensures a household's history remains ordered. The development deployment uses a single broker, so partitions demonstrate logical scalability.

5.4 Daily tariff source

Each file contains exactly one row for each of the twenty households. The fields are household_id, effective_date, tariff_rate, billing_tier and subsidy_flag. Four households are assigned the subsidized rate of LKR 32/kWh, twelve the standard rate of LKR 42/kWh and four the premium rate of LKR 48/kWh. A fixed seed makes assignment reproducible. These values are simulation assumptions only.

6 Stream Processing with Apache Spark

6.1 Validation and parsing

Spark reads Kafka records as a streaming Data Frame and converts the binary value to text. An explicit schema parses JSON and prevents uncontrolled type conventions. A record is accepted only when meter ID, household ID, zone and timestamp are present and both energy measures are non-negative. The pipeline then calculates grid import. In production, rejected records should be routed to a dead-letter topic with a reason.

6.2 Zone window aggregation

The operational query groups records by grid zone and five-minute event-time windows. For every zone/window pair it calculates total consumption, total solar generation, total grid import, reading count and renewable contribution:

Renewable Contribution = {Total Solar Generation/ Total Consumption} × 100

A ten-minute watermark allows bounded late arrival. Results are uploaded into *zone_energy_metrics* using window start, window end and grid zone as the logical key. This gives the dashboard complete intervals and prevents duplicate rows when a batch is retried.

6.3 Daily household aggregation

A second streaming query groups data by simulated date, household and grid zone. It produces daily totals and a reading count. The watermark delays final output until Spark has observed sufficiently later event time; this is why the producer must cross the day boundary and continue beyond it.

For each completed micro-batch, Spark opens a PostgreSQL transaction, uploads the daily rows, verifies household completeness and updates *daily_pipeline_status* to *energy_ready*. Both operations commit together. Airflow cannot see a ready flag without the associated energy data.

6.4 Checkpointing and restart behavior

The zone and daily queries use separate durable checkpoint locations. Checkpoints retain Kafka offsets, query metadata and state needed for window progress. Restarting Spark with the same volume continues from the stored position. Starting with an empty checkpoint and the configured latest Kafka offset intentionally consumes only records arriving after the query starts, so Kafka and Spark must be ready before the producer begins a clean demonstration.

7 Batch Orchestration with Apache Airflow

7.1 Tariff-ingestion DAG

The *tariff_ingestion* DAG is scheduled every five real minutes. A run discovers tariff CSV files and processes each date idempotently. Validation requires the exact expected columns, exactly twenty rows, twenty unique households, one effective date matching the file, valid tiers and non-negative rates. If validation succeeds, the DAG upserts tariff rows and updates tariff readiness in one transaction. An invalid file causes the task to fail. This prevents the system from silently producing partial billing input.

7.2 Daily-billing DAG

The *daily_billing* DAG is scheduled every real minute. The schedule is deliberately more frequent than a simulated day: it acts as a low-cost dispatcher that detects durable readiness quickly. The task selects every eligible date, not only the current wall-clock date. It changes a selected date from PENDING or FAILED to PROCESSING, joins the twenty daily energy rows to the twenty effective tariff rows, and calculates as shown.

Estimated bill (LKR) = Total Import from the grid × Tariff rate

Twenty bill rows and a COMPLETED state are committed atomically. On failure, the state becomes FAILED and error text is retained. Airflow task retries once after thirty seconds. if that still fails, a future scheduled run can select the failed date again. Upsert keys make retries safe.

7.3 Failure scenarios

Failure	Expected behavior	Recovery evidence
Airflow unavailable when Spark finalizes a day	Energy readiness remains stored. Billing occurs after Airflow restarts and scans.	Readiness row shows energy_ready=true and PENDING until the DAG runs.
Spark unavailable while producer continues	Kafka retains events subject to retention. Spark resumes from its checkpoint.	Kafka offsets increase; subsequent Spark micro-batches continue.
Tariff file missing	Energy remains ready, tariff remains false, and no bill is produced.	Readiness table shows the missing side explicitly.
Invalid tariff file	DAG fails before readiness is published.	Airflow task log contains validation error.
Billing retries	Existing logical bill rows are updated rather than duplicated.	Composite primary key and ON CONFLICT upsert enforce one bill per household/date.

8 Storage, Serving and Presentation

8.1 Relational storage model

Table	Grain and purpose	Primary key
zone_energy_metrics	One row per zone and completed five-minute event-time window.	window_start, window_end, grid_zone
household_daily_energy	One finalized energy summary per household and simulated date.	energy_date, household_id
tariff_reference	One tariff assignment per household and effective date.	household_id, effective_date
daily_pipeline_status	One readiness/billing state record per date.	energy_date
daily_household_billing	One calculated bill per household and date, including source measures and applied tariff.	energy_date, household_id

8.2 API layer

FastAPI exposes health, latest zone metrics, time-filtered zone history, renewable alerts, daily billing and household billing history. Parameterized SQL protects queries, response models define the contract and the API contains no write endpoints. The separation means the dashboard can be replaced without exposing PostgreSQL directly.

8.3 Dashboard

The Streamlit dashboard retrieves data only through FastAPI. The energy view allows the operator to select a date and time range, inspect consumption, solar, grid-import and renewable KPIs, compare zones and review low-renewable alerts. The billing view displays finalized household bills and tariff attributes for the selected date. A fifteen-second data cache reduces repeated API calls; the Refresh button clears the cache when a live update is required.

9 Observability, Reliability and Testing

9.1 What is observed and why

Signal	Measurement mechanism	Indications
Service lifecycle	Docker Compose status and container exit state	Which stateful component is running, healthy or failed?
Resource use	Docker CPU and memory statistics	Is the 8 GB workstation under memory pressure?
Kafka flow	Topic metadata, partition offsets and sampled messages in the Control Center	Are events reaching the topic and how are they distributed?
Spark progress	Micro-batch/database-write logs and PostgreSQL row counts	Are streaming batches being processed and persisted?
Batch progress	Airflow DAG/task states and logs	Did validation and billing succeed, retry or fail?
Cross-path readiness	daily_pipeline_status table	Which input is missing for a date and is billing complete?
Serving health	FastAPI /health and dashboard request behavior	Can consumers read the results?
Business condition	Renewable contribution threshold below 20% during daylight	Which latest zone requires operator attention?

9.2 Automated tests

The verified implementation contains 81 passing pytest tests. They cover API responses and validation, business calculations, configuration access, dashboard client behavior and Control Center commands, including non-interactive DAG unpausing. The run completed in 2.15 seconds. Tests validate component logic; the end-to-end demonstration additionally validates interactions between real containers.

9.3 Reliability controls

- Kafka separates producer and consumer lifetimes and retains the event log.
- Spark checkpoints retain offsets and state for restart.
- PostgreSQL transactions couple facts with their readiness signals.
- Primary keys and upserts make repeated streaming batches and DAG runs idempotent.
- Airflow stores task history, applies a retry and revisits failed dates.
- Docker health checks sequence service dependencies during startup.

10 Results and Evaluation

10.1 Demonstrated outputs

A complete run produces three categories of visible result. First, Kafka partition offsets and sampled JSON messages confirm ingestion. Second, Spark logs show numbered micro-batches and PostgreSQL contains zone windows and one daily household row per household. Third, Airflow task logs and the readiness table show tariff loading, date eligibility and completion of twenty household bills. The dashboard then presents the same stored results through the API.

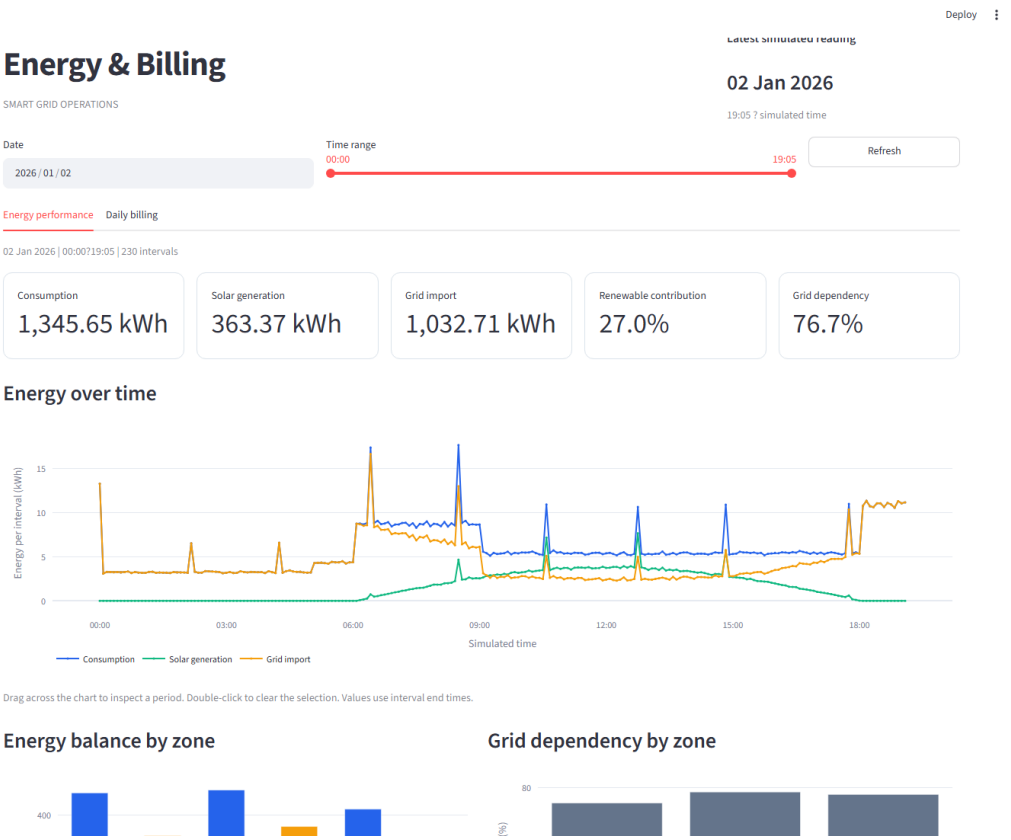


Figure 3: Energy-performance dashboard populated from PostgreSQL through FastAPI after stream processing.

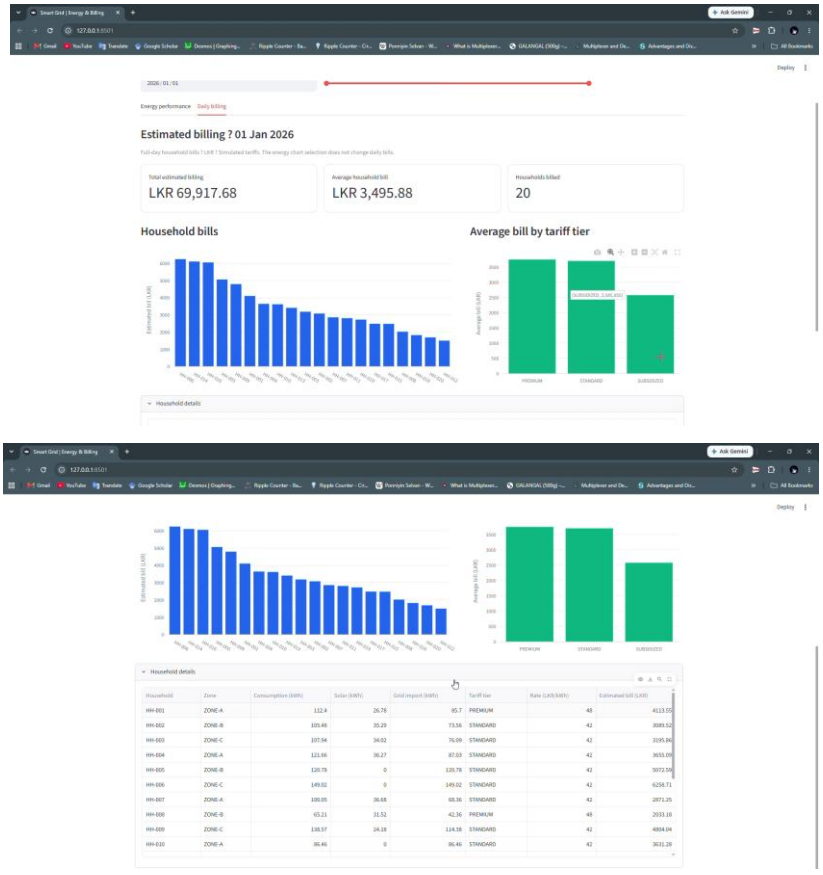


Figure 4: Bill values visualized in the dashboard after DAG runs

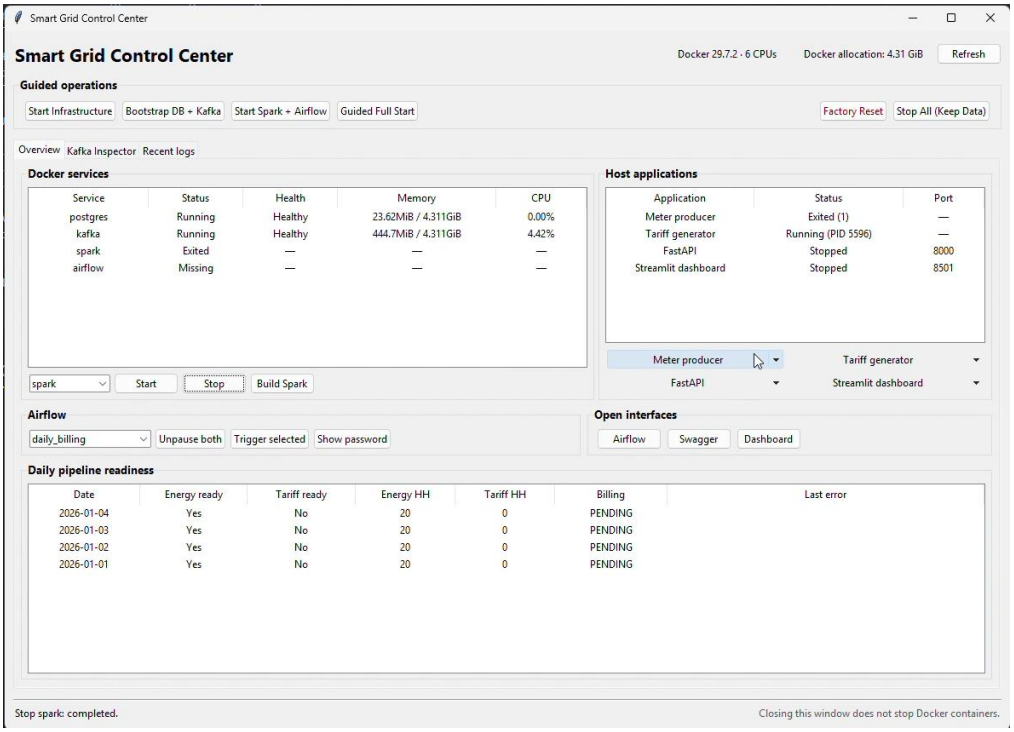


Figure 5: Control Center for System Monitoring and light weight control

10.2 Evaluation against requirements

Assessment area	Evidence	Evaluation
Architecture	Separate speed and batch paths with a documented PostgreSQL readiness boundary.	Meets Lambda/Kappa decision and architecture requirements.
Ingestion	Continuous keyed Kafka events and independent daily tariff files.	Meets the two-source requirement.
Processing	Event-time windows, watermark, daily aggregation, tariff validation and billing.	Meets streaming and batch requirements.
Storage/serving	Five constrained relational tables, six API routes and dashboard views.	Provides a usable system of record and presentation layer.
Observability	Logs, status, resource usage, offsets, readiness, API health and business alerts.	Covers technical and business-level signals.
Reproducibility	Compose configuration, versioned schema, locked Python dependencies and automated tests.	Supports repeatable local demonstration.

11 Limitations, Trade-offs and Production Improvements

11.1 Current limitations

- The single Kafka broker has no replication and is a single point of failure.
- PostgreSQL serves operational and analytical queries together. So, retention and long-range analytics are limited.
- Host-run Python processes are not supervised by Compose and require separate lifecycle management.
- Simulation profiles are simplified and do not represent certified meter physics, official tariffs, export payments or missing-device behavior.
- Filtered malformed meter events are not retained in a dead-letter topic.
- Observability uses logs and inspection tools rather than centralized metrics, tracing and alert delivery.
- Development credentials and open local ports are unsuitable for an exposed environment.
- The Spark daily output depends on watermark progress. So, a quiet stream may delay finalization without an explicit end-of-day control event.

11.2 Production-scale improvements

1. An improvement would be to deploy a multi-broker Kafka cluster with replication, authentication, authorization, encryption and a schema registry.
2. Placing raw immutable events and curated historical partitions in object storage using Parquet will be an improvement while keeping PostgreSQL for operational serving.
3. Adding a dead-letter topic and data-quality metrics for malformed, late and duplicate records would make the system more reliable and easier to troubleshoot.
4. Introducing explicit source event identifiers and end-of-day control events.
5. Using role-based API access, TLS, network isolation and audited credential rotation.

12 Conclusion

Our system demonstrates a complete smart-grid data platform with two source types and two processing modes. Kafka accepts a continuous, partitioned event stream. Spark turns event-time readings into zone metrics and final daily household energy. Airflow validates date-effective tariffs and calculates bills only after durable readiness proves both inputs are complete. PostgreSQL provides transactional synchronization and a clear serving model, while FastAPI and Streamlit make the results accessible.

13 Group Contribution Statement

Member	Student ID	Primary contribution	Additional contribution
Madhusankha T.G.S.A.	EG/2021/4656	Architecture, meter producer and Kafka ingestion	Documentation and end-to-end integration
Pathirathna A.H.	EG/2021/4705	Spark streaming, aggregations and PostgreSQL writes	Testing and demonstration
Horadagoda H.A.D.A.M.	EG/2021/4557	Airflow DAGs, API and dashboard	Observability and report preparation